

Table des matières

Labo 02 — Le bronze de l'historien et de la télémétrie.....	1
Avant de commencer	2
Étape 1 — Créer les dossiers du lac, puis convertir l'historien en Parquet, un dossier par jour.....	2
Étape 2 — Lire ce que les fichiers Parquet savent d'eux-mêmes.....	4
Étape 3 — Convertir la télémétrie en Parquet imbriqué.....	6
Étape 4 — Versionner les scripts, jamais le lac.....	8
Récapitulatif.....	8
Dépannage	9
Annexe — Tout le code, d'un seul tenant.....	10

Labo 02 — Le bronze de l'historien et de la télémétrie

Labo · Module 2/12 — Formation Data Engineering

Ce que vous allez faire. Ranger l'historien et la télémétrie dans le lac, en Parquet, avec des types déclarés et un dossier par jour livré ; puis mesurer ce que le format change, en octets et en secondes, et lire ce qu'un fichier Parquet sait de lui-même. Objectif de fin : deux scripts dans `01_de/sql/`, un bronze de 1,5 Mo pour l'historien et de 10,8 Mo pour la télémétrie, deux contrôles qui rendent 275 436 et 158 146 des deux côtés, et une requête qui ne lit qu'un fichier sur 92.

Deux terminaux servent dans ce labo, et chaque étape dit lequel. L'Anaconda Prompt, environnement `formations` actif, sert à Python, à Git et aux commandes du système. La ligne de commande DuckDB, qui s'ouvre par `duckdb` et se quitte par `.exit`, sert au SQL.

La liste de contrôle du cours :

- ☐ Le dépôt `01_de` du Cours 01 en place, `git status` propre
- ☐ Dossiers `plateforme\lac\bronze\usine` et `01_de\sql` créés
- ☐ `01_de/sql/bronze_historien.sql` lancé : 275 436 lignes des deux côtés, 92 fichiers, 1 468 497 octets
- ☐ Le schéma, les statistiques et l'élégage lus : Scanning Files: 1/92
- ☐ `01_de/sql/bronze_telemetrie.sql` lancé : 158 146 messages des deux côtés, 843 fichiers
- ☐ Les deux scripts validés dans le dépôt `01_de`

Avant de commencer

Ce cours ne demande rien de plus que le Cours 01 : le socle du parcours, les données de Boréal dans `Projet_Boreal/sources/`, et votre dépôt `01_de` avec son `.gitignore`. Aucun service à allumer. Si vous commencez la formation ici, faites d'abord l'étape 1 du Labo 01, qui crée le dépôt en quelques minutes.

Étape 1 — Créer les dossiers du lac, puis convertir l'historien en Parquet, un dossier par jour

Théorie : `Theorie_02.md`, sections 4, 5 et 6.

Le lac se range dans la plateforme, les scripts dans votre dépôt. Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir plateforme\lac\bronze\usine
mkdir 01_de\sql
```

Sous Windows, `mkdir` crée les dossiers intermédiaires d'un coup, mais il veut des barres obliques inverses : avec `/`, il prend le chemin pour une option. Le projet prend cette forme :

Projet_Boreal/	
├── sources/	ce que Boréal livre, généré à l'installation
├── plateforme/	
│ ├── lac/	
│ │ ├── bronze/	la donnée telle que reçue, rangée en Parquet
│ │ │ ├── usine/historien/	étape 1
│ │ │ └── telemetrie/	étape 3
└── 01_de/	
└── sql/	vos scripts, versionnés

L'argent du lac viendra au Cours 08, l'entrepôt au Cours 03. Les dossiers `historien` et `telemetrie`, DuckDB les créera lui-même ; leurs parents, non.

Le bronze de l'historien ne se tape pas dans la ligne de commande DuckDB : il s'écrit dans un fichier, un script, qu'on relancera à chaque nouvelle livraison et dont le Cours 05 fera une pièce du pipeline. La suite de l'étape se fait en trois temps : créer le fichier, y coller le code, puis le faire exécuter par DuckDB.

Si la ligne de commande DuckDB est encore ouverte, quittez-la (`.exit`). Dans l'Anaconda Prompt, depuis `Projet_Boreal`, créez le fichier et ouvrez-le dans le Bloc-notes :

```
notepad 01_de\sql\bronze_historien.sql
```

Le Bloc-notes répond qu'il ne trouve pas le fichier et propose de le créer : acceptez. Collez-y le contenu qui suit, en entier, puis enregistrez (Ctrl+S) et fermez le Bloc-notes :

```
-- 01_de/sql/bronze_historien.sql : L'historien, du CSV au Parquet partitionné par jour livré.
-- Lancement, depuis Projet_Boreal : duckdb -f 01_de/sql/bronze_historien.sql
```

```
COPY (
  SELECT
    TagName,
    Timestamp,
    Value,
    Quality,
    CAST(REPLACE(PARSE_FILENAME(filename, true), 'historien_', '') AS DATE) AS jour
  FROM read_csv(
    'sources/usine/historien/*.csv',
    header = true,
    filename = true,
    columns = {'TagName': 'VARCHAR', 'Timestamp': 'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'}
  )
) TO 'plateforme/lac/bronze/usine/historien'
(FORMAT parquet, COMPRESSION zstd, PARTITION_BY (jour), OVERWRITE);
```

```
-- Contrôle. Attendu : 275 436 des deux côtés.
```

```
SELECT
  (SELECT COUNT(*) FROM 'sources/usine/historien/*.csv') AS lignes_csv,
  (SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/*.parquet') AS lignes_parquet;
```

Ce que fait chaque morceau :

Morceau	Ce qu'il fait
columns = {...}	déclare le type de chaque colonne : plus de détection, plus de surprise
filename = true	ajoute le nom du fichier lu, d'où l'on tire le jour livré
CAST(REPLACE(PARSE_FILENAME(...)))	historien_2025-10-28.csv devient la date 2025-10-28
PARTITION_BY (jour)	un dossier par jour : jour=2025-10-28/
COMPRESSION zstd	la compression du bronze, plus compacte que snappy, le choix par défaut
OVERWRITE	vide le dossier avant d'écrire : relancer redonne le même bronze
le dernier SELECT	le contrôle, qui compare les lignes lues et les lignes écrites

À retenir — ce script est votre code. `bronze_historien.sql` vit dans votre dépôt, `01_de/sql`, et c'est vous qui l'écrivez. Les requêtes que ce labo vous fait taper dans la ligne de commande DuckDB, elles, ne se rangent nulle part : elles servent à regarder, pas à livrer.

Lancez-le, dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
duckdb -f 01_de/sql/bronze_historien.sql
```

Après quelques secondes, le contrôle s'affiche :

lignes_csv int64	lignes_parquet int64
275436	275436

Deux nombres égaux : le bronze contient tout l'historien. Deux nombres différents diraient que les sources ont changé depuis la conversion, ou que le script s'est arrêté en route. Relancez la même commande : le résultat est identique, parce qu'`OVERWRITE` refait le dossier de zéro.

Piège — un dossier parent qui manque. Si les deux `mkdir` du début de l'étape ont été sautés, DuckDB répond `IO Error: Failed to create directory "plateforme/lac/bronze/usine/historien"`. Il crée le dossier final d'un `COPY`, jamais ses parents.

Piège — relancer sans `OVERWRITE`. Retirez l'option et relancez : DuckDB refuse, avec `Directory "plateforme/lac/bronze/usine/historien" is not empty! Enable OVERWRITE option to overwrite files`. Remettez-la. Le Cours 03 fera de ce comportement une règle, l'idempotence.

Étape 2 — Lire ce que les fichiers Parquet savent d'eux-mêmes

Théorie : `Theorie_02.md`, sections 2, 3, 5 et 6.

Rouvrez DuckDB (`duckdb`). Le poids du bronze :

```
-- Attendu : 92 fichiers, 1 468 497 octets.  
SELECT COUNT(*) AS fichiers, SUM(size) AS octets  
FROM read_blob('plateforme/lac/bronze/usine/historien/**/*.parquet');
```

Les 13,7 Mo de CSV pesés au Cours 01 sont devenus 1,5 Mo, neuf fois moins, pour le même contenu. Les types, eux, sont ceux que le script a déclarés, plus la colonne de partition :

```
-- Attendu : 5 colonnes. TagName varchar, Timestamp timestamp, Value double, Quality integer, jour date.  
DESCRIBE SELECT * FROM 'plateforme/lac/bronze/usine/historien/**/*.parquet';
```

jour n'est écrite dans aucun fichier : DuckDB la lit dans le chemin, `jour=2025-10-28`. Le schéma que chaque fichier porte en lui :

```
-- Attendu : 5 lignes. duckdb_schema, la racine ; TagName BYTE_ARRAY (UTF8) ; Timestamp INT64
-- (TIMESTAMP_MICROS) ; Value DOUBLE ; Quality INT32 (INT_32).
SELECT name, type, converted_type
FROM parquet_schema('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
```

Un horodatage est un entier de 64 bits, compté en microsecondes ; une chaîne, une suite d'octets marquée UTF-8. Ces types voyagent avec le fichier : aucun outil n'a plus à les deviner. Les statistiques du même fichier :

```
-- Attendu : 4 lignes, une par colonne, toutes dans un groupe de 4 106 lignes, compressées en ZSTD.
SELECT path_in_schema, row_group_num_rows, compression, stats_min, stats_max,
       total_compressed_size, total_uncompressed_size
FROM parquet_metadata('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
```

Colonne	Minimum	Maximum	Octets compressés	Octets non compressés
TagName	ESS.BANC.REGIME	UTL.SEC.POINT_ROSEE	3 260	4 568
Timestamp	2025-10-27 00:00:00	2025-10-28 23:58:00	4 529	9 122
Value	-53.0	6856.33	12 598	32 742
Quality	0	192	94	132

Trois lectures. Le fichier du 28 octobre commence le 27 à minuit : ce sont les valeurs de départ, et elles restent dans le jour qui les a livrées. TagName couvre toutes les étiquettes, de la première à la dernière : un filtre sur une étiquette ne sautera rien dans ce fichier. Et Value, des nombres qui varient sans cesse, pèse plus que tout le reste, alors que Quality, presque toujours 192, tient en 94 octets.

La vitesse, maintenant. Activez le chronomètre de la ligne de commande, puis posez trois fois la même question, la vibration maximale du compresseur 2 :

```
.timer on

-- Attendu : 9.23 pour les trois requêtes.
SELECT MAX(Value) FROM 'sources/usine/historien/*.csv' WHERE TagName = 'UTL.CMP2.VIBRATION';

SELECT MAX(Value)
FROM read_csv('sources/usine/historien/*.csv', header = true,
             columns = {'TagName': 'VARCHAR', 'Timestamp': 'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'})
WHERE TagName = 'UTL.CMP2.VIBRATION';

SELECT MAX(Value) FROM 'plateforme/lac/bronze/usine/historien/*.parquet' WHERE TagName = 'UTL.CMP2.VIBRATION';
```

Sous chaque résultat, DuckDB affiche Run Time (s): real Sur le poste de l'auteur : environ 3 secondes pour les CSV aux types devinés, 0,6 seconde aux types déclarés, 0,016 seconde sur le bronze. Vos temps seront autres ; l'ordre de grandeur, non. Deviner les types coûte à chaque requête ; lire du texte ligne par ligne coûte encore ; le Parquet ne lit que deux colonnes compressées.

Enfin, l'élagage des partitions. EXPLAIN ANALYZE exécute la requête et montre ce qu'elle a lu :

```
-- Attendu : dans le bloc TABLE_SCAN, « Scanning Files: 1/92 » et « 4,106 rows ».  
EXPLAIN ANALYZE  
SELECT COUNT(*)  
FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'  
WHERE jour = DATE '2025-10-28';
```

Dans le dernier bloc du plan, celui du TABLE_SCAN, ces lignes :

```
|      File Filters:      |  
| (jour = '2025-10-28'::DATE) |  
|      Scanning Files: 1/92      |  
|      Total Files Read: 1      |
```

DuckDB a lu un fichier sur 92, sans ouvrir les autres : le filtre portait sur la partition, donc sur le chemin. Désactivez le chronomètre (.timer off) et quittez DuckDB (.exit).

Étape 3 — Convertir la télémétrie en Parquet imbriqué

Théorie : Theorie_02.md, sections 2, 4, 5 et 6.

La télémétrie demande plus de soin : chaque message porte une structure imbriquée, des horodatages avec leur décalage, et deux noms pour la même température du moteur, avant et depuis le micrologiciel 2.0 (le dictionnaire de Boréal les donne tous les deux). Créez le second script comme le premier, dans l'Anaconda Prompt (notepad 01_de/sql/bronze_telemetrie.sql), collez-y ce contenu, enregistrez et fermez le Bloc-notes :

```
-- 01_de/sql/bronze_telemetrie.sql : La télémétrie, du JSON Lines au Parquet imbriqué, par jour de réception.  
-- Lancement, depuis Projet_Boreal : duckdb -f 01_de/sql/bronze_telemetrie.sql
```

```
COPY (  
  SELECT  
    sujet,  
    recu_le,  
    qos,  
    message,  
    CAST(REPLACE(PARSE_FILENAME(filename, true), 'sorties_', '') AS DATE) AS jour  
FROM read_json(  
  'sources/telemetrie/*.jsonl',  
  format = 'newline_delimited',  
  filename = true,  
  columns = {  
    'sujet': 'VARCHAR',  
    'recu_le': 'VARCHAR',
```

```

        'qos': 'INTEGER',
        'message': 'STRUCT(
            message_id VARCHAR, sernr VARCHAR, micrologiciel VARCHAR,
            debut VARCHAR, fin VARCHAR,
            distance_km DOUBLE, heures_moteur DOUBLE,
            regime_moyen INTEGER, regime_max INTEGER,
            temp_moteur_max DOUBLE, temperature_moteur_max DOUBLE,
            temp_exterieure DOUBLE, tension_batterie_min DOUBLE,
            position STRUCT(lat DOUBLE, lon DOUBLE),
            codes VARCHAR[])'
    }
)
) TO 'plateforme/lac/bronze/telemetrie'
(FORMAT parquet, COMPRESSION zstd, PARTITION_BY (jour), OVERWRITE);

-- Contrôle. Attendu : 158 146 des deux côtés.
SELECT
    (SELECT COUNT(*) FROM read_json('sources/telemetrie/*.jsonl', format = 'newline_delimited',
                                     columns = {'sujet': 'VARCHAR'})) AS messages_json,
    (SELECT COUNT(*) FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet') AS messages_parquet;

```

La forme du message est déclarée en entier : un STRUCT dans lequel position est lui-même une structure et codes une liste de textes. `recu_le`, `debut` et `fin` restent du texte, décalage compris. Le contrôle, lui, lit l'archive en ne déclarant qu'une colonne : pour compter des lignes, c'est assez, et la détection n'a plus rien à rater. Lancez :

```
duckdb -f 01_de/sql/bronze_telemetrie.sql
```

Le contrôle doit rendre 158 146 des deux côtés. Rouvrez DuckDB :

```

-- Attendu : 843 fichiers, 10 817 501 octets : Les 884 jours, moins Les 41 jours sans message.
SELECT COUNT(*) AS fichiers, SUM(size) AS octets
FROM read_blob('plateforme/lac/bronze/telemetrie/*/*.parquet');

```

Les 70,6 Mo de JSON lines pesés au Cours 01 tiennent en 10,8 Mo. Un jour sans message ne donne aucun dossier : COPY ne partitionne que ce qui existe. Les deux noms de la température du moteur, chacun rempli par sa version du micrologiciel :

```

-- Attendu : 88 678 messages avec temp_moteur_max, 69 468 avec temperature_moteur_max, 158 146 en tout.
SELECT COUNT(message.temp_moteur_max) AS avant_2_0,
       COUNT(message.temperature_moteur_max) AS depuis_2_0
FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet';

```

COUNT d'une colonne ne compte que les valeurs présentes : chaque message remplit l'un des deux champs, jamais les deux. Réunir ces deux colonnes en une seule est une transformation, qui appartient à l'ingénieur analytique ; le bronze les garde telles que reçues. Les horodatages, tels que reçus aussi :

```
-- Attendu : 2025-12-15T00:25:22-05:00 et 2025-12-14T09:53:00-05:00.
SELECT recu_le, message.debut
FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet'
WHERE jour = DATE '2025-12-15'
ORDER BY recu_le
LIMIT 1;
```

Et le coût des petits fichiers, chronomètre activé :

```
.timer on

-- Attendu : 6 759 véhicules pour les deux requêtes.
SELECT COUNT(DISTINCT message.sernr) AS vehicules
FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true);

SELECT COUNT(DISTINCT message.sernr) AS vehicules
FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet';
```

Sur le poste de l'auteur : environ 0,6 seconde pour le JSON, 0,15 seconde pour le bronze. Le Parquet gagne encore, mais bien moins que sur l'historien : ses 843 fichiers pèsent 13 Ko en moyenne, et les ouvrir un à un coûte plus que les lire. L'exercice P5 mesure ce que rendrait un seul fichier. Tapez `.timer off`, puis `.exit`.

`union_by_name = true` règle au passage le piège vu à l'étape 3 du Labo 01 : DuckDB lit alors la structure de chaque fichier, au lieu de s'en remettre aux premiers, qui sont vides.

Étape 4 — Versionner les scripts, jamais le lac

Les deux scripts sont du code : ils vont dans le dépôt. Le bronze, lui, vit dans `plateforme/`, hors du dépôt, et se reconstruit en relançant les scripts. Depuis `Projet_Boreal` :

```
cd 01_de
git add sql
git commit -m "Cours 02 : bronze de l'historien et de la télémétrie en Parquet"
git status
cd ..
```

`git status` doit répondre que l'arbre de travail est propre. Le dépôt contient maintenant le `.gitignore` du Cours 01 et les deux scripts.

Récapitulatif

Vous avez...	Preuve
préparé le lac	plateforme\lac\bronze\usine et 01_de\sql existent
écrit le bronze de l'historien	275 436 lignes des deux côtés ; 92 fichiers, 1 468 497 octets
lu ce que le Parquet sait de lui-même	un groupe de 4 106 lignes pour le 28 octobre, ses minimums et maximums ; Scanning Files: 1/92
écrit le bronze de la télémétrie	158 146 messages des deux côtés ; 843 fichiers, 10 817 501 octets ; 88 678 et 69 468 messages par version de la température
versionné les scripts	git status propre dans 01_de

Prochaine étape (Cours 03). Le bronze garde des fichiers ; l'ingénieur analytique attend des tables. Au Cours 03, les extraits de l'ERP et du portail des garanties entrent dans la couche RAW d'un entrepôt DuckDB, en texte, par un chargement qu'on peut rejouer sans rien doubler.

Dépannage

Symptôme	Cause probable	Correctif
DuckDB : No files found that match the pattern	duckdb lancé ailleurs que dans Projet_Boreal	quitter DuckDB (.exit), se placer dans le projet, relancer duckdb
Après Entrée, DuckDB n'affiche rien et attend la suite	la requête n'a pas son point-virgule final	taper ; , puis Entrée
mkdir répond que la syntaxe est incorrecte	des barres obliques / au lieu de \	mkdir plateforme\lac\bronze\usine
IO Error: Failed to create directory "plateforme/lac/bronze/usine/historien"	les deux mkdir de l'étape 1 n'ont pas été faits : les dossiers parents manquent	créer les dossiers, puis relancer
Directory "... " is not empty! Enable OVERWRITE option to overwrite files	l'option OVERWRITE a disparu du script	la remettre dans la parenthèse du COPY
IO Error: Failed to open file "01_de/sql/bronze_historien.sql"	le terminal n'est pas dans Projet_Boreal, ou le script porte un autre nom : créé par l'Explorateur, il s'appelle peut-être bronze_historien.sql.txt	se placer dans Projet_Boreal ; dir 01_de\sql montre le vrai nom ; le renommer au besoin (ren)

Symptôme	Cause probable	Correctif
Le bronze existe, mais 01_de\sql est vide	le code des étapes 1 et 3 a été tapé dans la ligne de commande DuckDB au lieu d’être enregistré dans un fichier	créer les deux scripts avec notepad (étapes 1 et 3) et les lancer : ce sont eux qui vont dans le dépôt, et le bronze se refait à l’identique
No files found that match the pattern "plateforme/lac/bronze/..."	le script n’a pas tourné, ou DuckDB n’est pas lancé depuis Projet_Boreal	relancer le script ; relancer duckdb depuis le projet
Binder Error: Referenced table "message" not found!	l’archive a été lue d’un seul motif, sans déclarer sa forme : une seule colonne json	déclarer les colonnes, comme le script, ou ajouter union_by_name = true
Le contrôle rend deux nombres différents	les sources ont été régénérées depuis la conversion, ou le script s’est arrêté en route	relancer le script : OVERWRITE refait le bronze de zéro
Le contrôle de l’historien rend 2 133 145 lignes côté CSV	le générateur a tourné avec --historien complet	c’est cohérent si c’est voulu : relancer le script ; sinon, régénérer sans option, puis relancer
Conversion Error sur Timestamp ou Value	un fichier de l’historien a été modifié à la main	régénérer les sources : on ne modifie jamais un export

Annexe — Tout le code, d’un seul tenant

Pour rejouer le labo sans relire les explications. Les commandes de terminal se tapent dans l’Anaconda Prompt, depuis Projet_Boreal ; les requêtes, dans la ligne de commande DuckDB, lancée depuis le même dossier.

1 — Les dossiers (Anaconda Prompt)

```
mkdir plateforme\lac\bronze\usine
mkdir 01_de\sql
```

2 — Les deux scripts (créer 01_de\sql\bronze_historien.sql et 01_de\sql\bronze_telemetrie.sql avec le contenu des étapes 1 et 3, puis, dans l’Anaconda Prompt)

```
duckdb -f 01_de/sql/bronze_historien.sql
duckdb -f 01_de/sql/bronze_telemetrie.sql
```

3 — Ce que le Parquet sait de lui-même (DuckDB)

```
SELECT COUNT(*) AS fichiers, SUM(size) AS octets FROM read_blob('plateforme/lac/bronze/usine/historien/*/*.parquet');
DESCRIBE SELECT * FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet';
```

```

SELECT name, type, converted_type FROM parquet_schema('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
SELECT path_in_schema, row_group_num_rows, compression, stats_min, stats_max, total_compressed_size, total_uncompressed_size FROM
parquet_metadata('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
.timer on
SELECT MAX(Value) FROM 'sources/usine/historien/*.csv' WHERE TagName = 'UTL.CMP2.VIBRATION';
SELECT MAX(Value) FROM read_csv('sources/usine/historien/*.csv', header = true, columns = {'TagName': 'VARCHAR', 'Timestamp':
'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'}) WHERE TagName = 'UTL.CMP2.VIBRATION';
SELECT MAX(Value) FROM 'plateforme/lac/bronze/usine/historien/**/*.parquet' WHERE TagName = 'UTL.CMP2.VIBRATION';
EXPLAIN ANALYZE SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/**/*.parquet' WHERE jour = DATE '2025-10-28';
SELECT COUNT(*) AS fichiers, SUM(size) AS octets FROM read_blob('plateforme/lac/bronze/telemetrie/**/*.parquet');
SELECT COUNT(message.temp_moteur_max) AS avant_2_0, COUNT(message.temperature_moteur_max) AS depuis_2_0 FROM
'plateforme/lac/bronze/telemetrie/**/*.parquet';
SELECT recu_le, message.debut FROM 'plateforme/lac/bronze/telemetrie/**/*.parquet' WHERE jour = DATE '2025-12-15' ORDER BY recu_le
LIMIT 1;
SELECT COUNT(DISTINCT message.sernr) AS vehicules FROM read_json('sources/telemetrie/*.jsonl', union_by_name = true);
SELECT COUNT(DISTINCT message.sernr) AS vehicules FROM 'plateforme/lac/bronze/telemetrie/**/*.parquet';
.timer off
.exit

```

4 — Le dépôt (Anaconda Prompt, depuis Projet_Boreal)

```

cd 01_de
git add sql
git commit -m "Cours 02 : bronze de l'historien et de la télémétrie en Parquet"
git status
cd ..

```